Práctica 2 parte 2: Calculadora con Apache Thrift

Desarrollo de Sistemas Distribuidos. Grupo DSD-3.

Grado en Ingeniería Informática

Curso 2025-2026.

ETSIIT. Universidad de Granada.





UNIVERSIDAD DE GRANADA

Autor:
Ángel Rodríguez Faya

Profesor:
José Ángel Segura Muros

Fecha de entrega:
27 de marzo de 2026

Índice

1. Introducción. **2**

2. Requerimientos. **2**
2.1. Requerimientos básicos. 2
2.2. Requerimientos avanzados. 2

3. Componentes. **4**
3.1. Calculadora en Apache Thrift. 5
3.2. Servidor. 7
3.2.1. Servidor en Python. 7
3.2.2. Servidor en Java. 8
3.3. Cliente en Python. 8

4. Guía de uso de la calculadora distribuida. **10**
4.1. Instalación de Thrift. 10
4.2. Proceso para utilizar Thrift. Compilación y Ejecución. 12
4.2.1. Compilar el archivo `calculadora.thrift` 12
4.2.2. Arrancar el Servidor en Python 12
4.2.3. Arrancar el Servidor en Java 12
4.2.4. Ejecutar el Cliente en Python 13
4.3. Ejemplos de uso. 13
4.3.1. Ejemplo nº1. Operación básica. División 14
4.3.2. Ejemplo nº2. Operación básica. Área de un Círculo. . . 15
4.3.3. Ejemplo nº3. Operación con matrices 17
4.3.4. Ejemplo nº4. Procesamiento de un lote de operaciones 19

1. Introducción.

En esta práctica tenemos que volver a implementar la calculadora de la 1ª parte de la práctica 2, que realizaba 4 operaciones básicas (suma, resta, multiplicación y división) más 3 operaciones para calcular áreas de tres figuras geométricas. Pero, en este caso, en lugar de implementarlo con `Sun RPC` tenemos que hacerlo con `Apache Thrift`.

`Apache Thrift` es un *framework* de desarrollo de software de código abierto, creado originalmente por Facebook y gestionado por la *Apache Software Foundation*, que permite la comunicación eficiente y directa entre servicios escritos en diferentes lenguajes de programación. Además tiene un motor de generación de código para crear servicios **RPC** *(Remote Procedure Call)* que funcionan de manera eficiente y sin problemas entre C++, Java, Python, etc., facilitando así el desarrollo de sistemas distribuidos. Para más información, podemos visitar la página web oficial de Apache Thrift.

2. Requerimientos.

El propósito de este apartado es detallar los requisitos (tanto básicos como avanzados) que se nos pedían para realizar la práctica, además de hacer más comprensible el documento, ya que de un simple vistazo se puede ver lo que se implementa o lo que no este proyecto de la calculadora distribuida.

2.1. Requerimientos básicos.

Como requisitos básicos se nos pedía que realizara las 4 operaciones básicas descritas en la 1ª parte de la práctica, además de añadir el cálculo del área de cuadrados, triángulos y círculos. Estas operaciones **se encuentran implementadas y disponibles.**

2.2. Requerimientos avanzados.

Como requerimientos avanzados que se nos proporcionaban de ejemplo para añadir funcionalidades a nuestra calculadora se encuentran:

- **Enviar una estructura de operaciones y operandos para que se realicen a la vez.** Más adelante, en el apartado de componentes podremos ver que hay definida una estructura llamada `Operación` que encapsula los dos operandos y el operador para realizar la correspondiente operación, aunque también se tiene en cuenta que hay operaciones que sólo necesitan un operador. En el menú de la calculadora

lo veremos implementado como **Procesar un lote de operaciones**, al que se le pasa una colección de objetos de la estructura `Operación`, calcula el resultado de cada uno de ellos, y luego nos devuelve todos los resultados a la vez.

- **Operaciones con matrices.** Se puede realizar la **suma** y **multiplicación** de dos matrices, que se piden al usuario por la terminal. No hay implementadas operaciones con vectores como tal, pero el motivo era por simplificar y no sobrecargar tanto el menú. **Aunque se pueden realizar operaciones con vectores manteniendo una de las componentes fijas** (por ejemplo, para sumar dos vectores de n *elementos*, introducimos dos matrices de dimensiones *1 x n*, es decir, tienen 1 sola fila y n columnas).

- **Gestión de excepciones**. Se ha tenido en cuenta a la hora de implementar las operaciones ya que si se produce una excepción, bien porque estamos introduciendo un valor incorrecto (por ejemplo un carácter donde pide un número), o bien porque no hemos podido establecer una conexión con el servidor; todas ellas se han tratado de capturar y manejar para que, en lugar de que nuestro programa finalice de forma repentina y no sepamos porque, muestre donde ha ocurrido la excepción y sepamos reconocer de donde viene el problema. Además, en el fichero `calculadora.thrift` donde definimos los tipos de datos y los servicios, también se ha definido una excepción para que se lance cuando se intenta realizar una operación que no es posible (como por ejemplo dividir entre 0 o calcular el logaritmo de un número negativo).

- **Utilizar distintos lenguajes para crear el cliente y/o el servidor.** En mi caso, he implementado el cliente en Python y el servidor en Java y en Python. El objetivo de esto es tratar de aprender y también demostrar que no hace falta que el cliente y el servidor estén escrito en el mismo lenguaje, si no que puede estarlo en diferentes lenguajes e igualmente pueden trabajar juntos gracias a herramientas como `Apache Thrift` que hacen que esto sea posible.

- **Cualquier otra operación que se considere interesante.** Además de las operaciones ya mencionadas anteriormente, se han añadido las siguientes:

  - Potencia de x elevado a y.
  - Raíz cuadrada de x.
  - Logaritmo en base 10 de x.

- Logaritmo neperiano (o natural) de x.
- Seno de x grados
- Coseno de x grados
- Tangente de x grados.
- Además, exceptuando las matrices que trabajan con números enteros, el resto de operaciones de la calculadora se realizan con números en coma flotante (también llamados `double` o `float` dependiendo del lenguaje de programación.)

3. Componentes.

En esta sección veremos cada uno de los componentes que forman este programa/práctica.

Además, en la siguiente imagen podemos ver el diagrama de clases en UML. También se encuentra disponible en formato `.png` en `memoria/imagenes_necesarias/diagrama_de_clases.png`

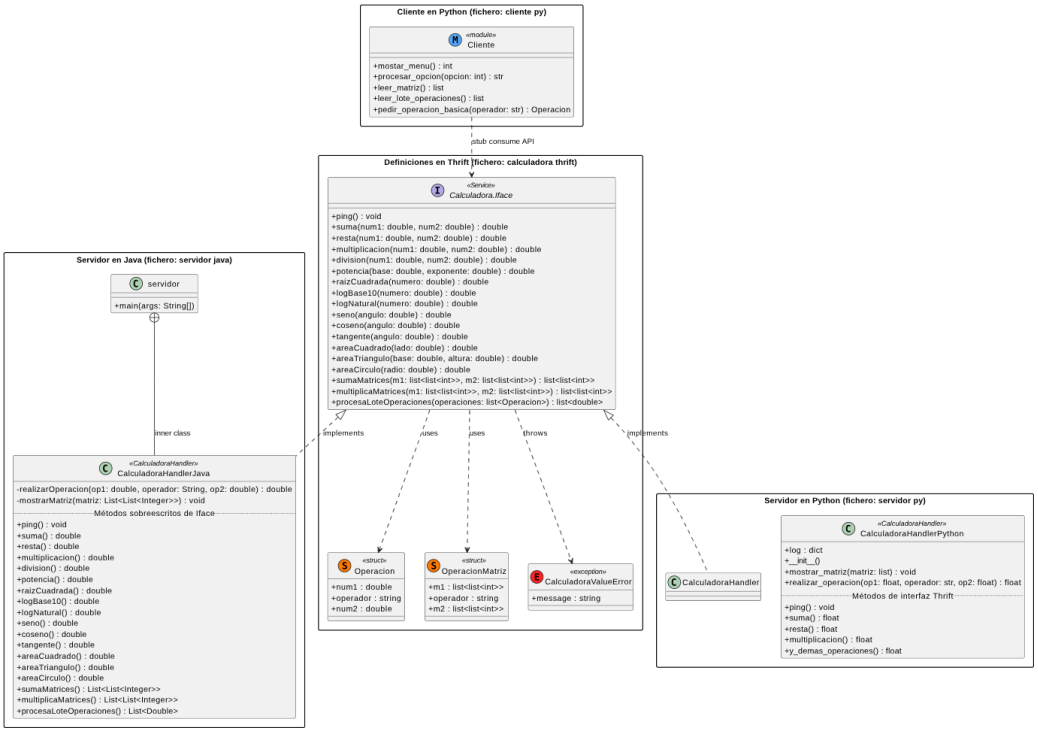


Figura 1: Diagrama de clases.

3.1. Calculadora en Apache Thrift.

Es el fichero **IDL** (*Interface Definition Language*). Se encuentra en el archivo `calculadora.thrift` y su contenido es:

```
/* calculadora.thrift */
namespace java calculadora // Para poder generar código Java

struct Operacion {
    1: double num1,
    2: string operador,
    3: double num2
}

struct OperacionMatriz {
    1: list<list<i32>> m1,
    2: string operador,
    3: list<list<i32>> m2
}

exception CalculadoraValueError{
    1: string message
}

service Calculadora{
    void ping(),

    double suma(1:double num1, 2:double num2),
    double resta(1:double num1, 2:double num2),
    double multiplicacion(1:double num1, 2:double num2),
    double division(1:double num1, 2:double num2) throws (1: CalculadoraValueError ex),

    double potencia(1:double base, 2:double exponente),
    double raizCuadrada(1:double numero) throws (1: CalculadoraValueError ex),
    double logBase10(1:double numero) throws (1: CalculadoraValueError ex),
    double logNatural(1:double numero)  throws (1: CalculadoraValueError ex),
    double seno(1:double angulo),
    double coseno(1:double angulo),
    double tangente(1:double angulo),

    double areaCuadrado(1:double lado),
    double areaTriangulo(1:double base, 2:double altura),
    double areaCirculo(1:double radio),

    list<list<i32>> sumaMatrices(1: list<list<i32>> m1, 2: list<list<i32>> m2)
        throws (1: CalculadoraValueError ex),

    list<list<i32>> multiplicaMatrices(1: list<list<i32>> m1, 2: list<list<i32>> m2)
```

```
44          throws (1: CalculadoraValueError ex),
45
46      list<double> procesaLoteOperaciones(1: list<Operacion> operaciones)
47          throws (1: CalculadoraValueError ex)
48  }
```

El archivo `calculadora.thrift` podríamos definirlo como el ***contrato de comunicación*** que firman las partes implicadas en nuestro sistema distribuido, el cliente (que recordamos que estaba implementado en Python) y el servidor (implementado tanto en Python como en Java). En él se establece tres cosas: qué se puede hacer, cómo se puede hacer o cómo se debe pedir y qué vamos a recibir, sin importarle el lenguaje de programación en el que se vaya a desarrollar después. Es por eso que tenemos el cliente y el lenguaje implementados en diferentes lenguajes de programación.

Vamos a distingir 4 partes importantes dentro de nuestro archivo `calculadora.thrift`:

- **Espacios de nombres (*Namespaces*)**. Le dicen al compilador de Thrift cómo debe organizar el código cuando lo traduzca. En nuestro archivo tenemos `namespace java calculadora` que es necesario para poder generar el código en Java. En este caso, se le indica al compilador que meta el código de Java en un paquete llamado "`calculadora`".

- **Excepciones especializadas *(Exceptions)***. Nos permiten crear errores a medida tal que, si el servidor detecta que no es posible realizar una operación (por ejemplo, dividir entre cero), le envía este error o excepción al cliente y este ya sabe que se trata de un error producido por la imposibilidad de realizar una acción. En nuestro código, la excepción personalizada es `except CalculadoraValueError`.También existen otras opciones predefinidas en los lenguajes de programación.

- **Estructuras de datos (*Structs*)**. Funcionan de igual manera que en cualquier otro lenguaje de programación, sirven para empaquetar datos. En nuestro caso, se utilizan para empaquetar los datos y enviarlos por la red de golpe. En el código tenemos dos estructuras: `struct Operacion` que empaqueta una operación normal con sus dos operandos (o uno en caso de una operación unaria) y el operador; y luego encontramos `struct OperacionMatriz` que almacena dos matrices y el operador.

- **Servicio *(Service)***. Aquí es donde declaramos todas las funciones de la calculadora. Para cada función definimos: el tipo de dato que devuelve, el nombre de la función, los parámetros que recibe y las excepciones que pueda lanzar.

3.2. Servidor.

El servidor representa el núcleo lógico y computacional de nuestro sistema distribuido. Su función principal es mantenerse a la escucha en un puerto de red específico (oara esta práctica,es el puerto 9090), recibir las peticiones de operaciones matemáticas que el envía el cliente, procesarlas y devolver el resultado correspondiente utilizando el protocolo establecido por *Apache Thrift*.

Es fundamental definir el concepto de ***Handler*** (Manejador) para entender la arquitectura interna de los servidores implementados con *Apache Thrift*. Al compilar el archivo `.thrift`, la herramienta genera automáticamente una interfaz (habitualmente denominada `Iface` en el código generado). Esta interfaz actúa contiene la cabecera de todas las funciones declaradas en el contrato (el archivo `calculadora.thrift`), pero, como se trata de una interfaz, no están implementadas. El ***Handler*** es la clase que nosotros debemos desarrollar manualmente para implementar dicha interfaz. Podríamos decir que el *Handler* es el cerebro de la calculadora, pues es el bloque de código donde programamos la lógica y definimos qué operaciones aritméticas o de control se ejecutan realmente cuando el servidor recibe una llamada remota.

Aunque el archivo `.thrift` es único e invariable, la implementación del servidor y su *Handler* se adaptan a las particularidades y paradigmas de cada lenguaje de programación. A continuación, vamos a ver las peculiaridades de las dos implementaciones que tiene el servidor en este proyecto:

3.2.1. Servidor en Python.

Se encuentra en el archivo `servidor.py`. Al ser Python un **lenguaje interpretado y de tipado dinámico**, permite una implementación del servidor más ágil y directa. Es por eso que, la definición de la clase del *Handler* y la configuración del motor del servidor (`TSimpleServer`) están en un mismo script . Al ser de tipado dinámico, la manipulación de estructuras complejas (como las matrices que las he representado como listas anidadas), es flexible y no requiere declaraciones de tipo explícitas, lo que agiliza el desarrollo.

También tiene una gran ventaja respecto a la implementación en Java, y es la **gestión de dependencias y ejecución**. Al no existir un proceso de compilación previo, la integración con el código autogenerado por Thrift se resuelve en tiempo de ejecución modificando las variables de entorno del script (mediante `sys.path.append('gen-py')`) para indicar al intérprete dónde localizar el directorio `gen-py`, generado automáticamente

por `Thrift`. Una vez hecho esto, el servidor se levanta ejecutando el script directamente. Esto lo veremos en el punto *4. Guía de uso de la calculadora distribuida.*

3.2.2. Servidor en Java.

A diferencia de Python, **Java es un lenguaje compilado y de tipado estático**. Esto hace que se añada un nivel superior de robustez y control de errores en tiempo de compilación, pero exige una arquitectura y configuración más rigurosas, es el precio a pagar.

Respecto a la **estructura del *Handler***, debe implementarse como una clase formal, tal y como indican los estándares de la programación orientada a objetos en Java. Para centralizar el código fuente en un único archivo (`servidor.java`), el *Handler* se ha diseñado como una clase anidada estática (`public static class CalculadoraHandler`). Además, requiere la declaración explícita de las **excepciones personalizadas** importadas del contrato (usando la cláusula `throws`).

Debido al tipado estricto, la **implementación** de la lógica matemática exige el uso de las colecciones formales del lenguaje. Por ejemplo, el tratamiento de matrices obliga a trabajar con la interfaz `List` y su implementación `ArrayList (List<List<Integer>>)`, garantizando la integridad de los datos en tiempo de compilación.

A diferencia de Python, el servidor en Java requiere un **paso explícito de compilación** (`javac`) y otro de ejecución (`java`). Durante ambos procesos, es de carácter obligatorio gestionar el *Classpath* (ruta de clases) para enlazar manualmente el código autogenerado en el directorio `gen-java` junto con las librerías binarias externas requeridas por *Apache Thrift*.

3.3. Cliente en Python.

El cliente actúa como el punto de entrada o interfaz para el usuario final en nuestro sistema distribuido. Se encuentra implementado en el archivo `cliente.py`. Su responsabilidad principal es: recoger las peticiones del usuario, enviarlas al servidor remoto y mostrar los resultados obtenidos.

Para entender el funcionamiento del cliente, podemos dividir su arquitectura en los siguientes puntos clave:

- **Pila de comunicación (*Transport* y *Protocol*):** Para que el cliente pueda establecer contacto con el servidor, configura una conexión de red apuntando a la dirección anfitriona (`localhost`) y al puerto (`9090`) mediante las clases `TSocket` y `TBufferedTransport`. Además, utiliza

un protocolo binario (`TBinaryProtocol`) para empaquetar y serializar los datos de forma eficiente antes de enviarlos por la red.

- **El concepto de *Stub* (o Proxy):** Al igual que en el servidor hablábamos del *Handler* para implementar la lógica, en el lado del cliente *Thrift* genera una clase denominada `Client` (también conocida como *stub* o *proxy*). Este objeto presenta exactamente las mismas funciones definidas en `calculadora.thrift`. Gracias a este *stub*, podemos realizar una invocación a través de la red, como `client.suma(2, 3)`, se escribe en el código exactamente igual que si estuviéramos llamando a una función local, ocultando al programador toda la complejidad subyacente de la red.

- **Interfaz interactiva (Menú):** El cliente está diseñado como una aplicación de consola iterativa. Cuenta con funciones auxiliares encargadas de mostrar por pantalla un menú principal donde se listan todas las operaciones matemáticas y utilidades disponibles (como el procesamiento de un lote de operaciones). El programa se mantiene en un bucle continuo hasta que el usuario decide salir explícitamente (`opcion = 0`).

- **Control de flujo y ejecución:** Una vez que el usuario introduce la opción deseada por teclado, el flujo del programa se gestiona mediante una estructura de control condicional (un bloque `match-case` o también conocido como `if-elif-else`). Según la operación seleccionada, el cliente pide al usuario los operandos necesarios por consola, empaqueta la información y realiza la llamada a la función remota correspondiente del *stub*.

- **Manejo de errores:** Es fundamental destacar que las llamadas al servidor están protegidas mediante bloques `try...except`. Esto permite capturar nuestra excepción personalizada `CalculadoraValueError` proveniente del servidor. De este modo, si ocurre un error matemático (como una división por cero), se informa al usuario por pantalla sin que la ejecución del cliente se interrumpa de forma abrupta.

4. Guía de uso de la calculadora distribuida.

En esta sección se va a mostrar un ejemplo de uso de la calculadora distribuida en **Ubuntu** (Linux), en concreto la **versión 24.04 *LTS***. Ten en cuenta que si se trata de un Sistema Operativo distinto, muy probablemente los comandos sean diferentes.

Para ello, se asumirá que partimos de una instalación limpia donde no tenemos instalado el compilador de *Thrift* ni las librerías necesarias.

Nota: el siguiente proceso ilustra tal y como yo lo he hecho en mi ordenador, por lo que, siguiendo estos mismos pasos que detallo a continuación, me ha funcionado correctamente.

4.1. Instalación de Thrift.

Deberemos de seguir los siguientes pasos:

1. **Instalar el compilador de** `Thrift`. El compilador es la herramienta central que nos permite procesar el archivo `calculadora.thrift` y generar automáticamente el código base para el cliente y el servidor. Para instalarlo ejecutamos el siguiente comando:

   ```
   $ sudo apt-get install thrift-compiler
   ```

   Para verificar que la instalación ha sido correcta, basta con ejecutar `thrift -version` en la terminal.

2. **Instalación de dependencias para Python.** Una vez instalado el compilador, nuestro entorno Python necesita las librerías nativas de Thrift para poder utilizar las clases de red y protocolo (como `TSocket`). Para instalar este paquete, tenemos que seguir una serie de pasos:

   a) Vamos a necesitar el gestor de paquetes de Python (`pip`). Lo podemos instalar utilizando el comando:

   ```
   $ sudo apt install python3-pip
   ```

   b) Ahora, vamos a **crear un entorno virtual** *(virtual enviroment)*, necesario para poder utilizar `pip`. Con el siguiente comando creamos un entorno virtual en una carpeta llamada `entorno_calculadora`

   ```
   $ python -m venv entorno_calculadora
   ```

Si nos salta un **error** diciendo que no tenemos instalado `venv`, lo podemos instalar utilizando el comando `$ sudo apt install python3.12-venv`. Y ya volvemos a ejecutar el comando para crear el entorno.

c) Para **activar el entorno** (es decir, acceder a ese directorio aislado) ejecutamos:

```
$ source entorno_calculadora/bin/activate
```

Y veremos como en la terminal, al principio de la línea aparece `(entorno_calculadora)` antes de neustro nombre de usuario, eso indica que hemos activado el entorno virtual y estamos dentro.

d) Ahora ya podemos **instalar los paquetes de Thrift en Python**. Ejecutamos el siguiente comando:

```
$ pip install thrift
```

e) Una vez que se hayan instalado satisfactoriamente, salimos del entorno ejecutando:

```
$ deactivate
```

3. **Instalar las bibliotecas de Thrift necesarias para Java**. Necesitaremos descargas las librerías necesarias para poder ejecutar el código en Java. Necesitaremos instalar dos librerías, pero será necesario que sepamos la versión de Thrift que tenemos instalada, para ello ejecutamos el comando: `$ thrift -version`. Una vez que sepamos la versión, ya podemos instalar las librerías:

   a) La primera es la librería `Thrift version 0.xx.0`. Para descargarla vamos a la página oficial Maven Central Repository - libthrift y pinchamos sobre nuestra versión de Thrift. Nos deberemos de asegurar de que es un archivo `.jar` y que no contiene palabras extras como `javadoc`

   b) La segunda librería es para que Thrift pueda imprimir por la terinal y no nos proteste. Se llama `SLF4J API:`. Para descargarla vamos a la página oficial Maven Central - slf4j-api 1.7.36 y descargamos el archivo: `slf4j-api-1.7.36.jar`.

   c) La tercera y última librería se llama `Javax Annotation`. Para descargarla vamos a Maven Central - javax.annotation-api 1.3.2 y descargamos el archivo: `javax.annotation-api-1.3.2.jar`.

Finalmente, creamos una carpeta llamada `lib` y metemos dentro estas tres librerías. **Es muy importante** que esta carpeta o directorio esté a la misma altura que el servidor implementado en Java (`servidor.java`)

4.2. Proceso para utilizar Thrift. Compilación y Ejecución.

4.2.1. Compilar el archivo `calculadora.thrift`

Necesitaremos compilar el archivo dos veces para generar los archivos necesarios para Python y para Java. Para ello debemos de ejecutar los siguientes dos comandos:

```
$ thrift -gen py calculadora.thrift
```

```
$ thrift -gen java calculadora.thrift
```

Al ejecutar estos comandos se nos generarán dos directorios con el código autogenerado por el compilador de Thrift: `gen-py` y `gen-java`

4.2.2. Arrancar el Servidor en Python

Si queremos ejecutar el servidor implementado en Python abrimos una terminal y ejecutamos el siguiente comando:

```
$ python3 servidor.py
```

4.2.3. Arrancar el Servidor en Java

En este caso deberemos de compilar y ejecutar el código, puesto que Java s un lenguaje compilado y no interpretado como Python.

Para **compilar** el servidor abrimos una terminal y ejecutamos el siguiente comando:

```
$ javac -d bin -cp ".:lib/*" -sourcepath gen-java servidor.java
```

Para **arrancar** el servidor ejecutamos:

```
$ java -cp "bin:gen-java:lib/*" servidor
```

4.2.4. Ejecutar el Cliente en Python

Para ejecutar el cliente en Python debemos de **abrir una terminal distinta a la del servidor** y ejecutar el siguiente comando:

```
$ python3 cliente.py
```

4.3. Ejemplos de uso.

Voy a mostrar 4 ejemplos de uso donde se realizarán tres opciones distintas: dos operaciones básicas, una operación con matrices y un procesamiento de un lote de operaciones.

Cabe destacar que **los servidores en sus dos implementaciones funcionan exactamente igual**, por lo que los ejemplos funcionan igual tanto si arrancamos el servidor en Python como si arrancamos el servidor de Java.

El menú de opciones de la calculadora, que encontraremos en el cliente es el siguiente:

```
==== CALCULADORA DISTRIBUIDA usando Apache Thrift ============================
Opciones disponibles:

  0. Salir de la aplicación.
  1. Suma                                   '+'
  2. Resta                                  '-'
  3. Multiplicación                         'x'
  4. División                               '/'
  5. Potencia                               '^'
  6. Raíz cuadrada                          'sqrt'
  7. Logaritmo base 10                      'log10'
  8. Logaritmo natural                      'ln'
  9. Seno                                   'sin'
  10. Coseno                                'cos'
  11. Tangente                              'tan'
  12. Área de un cuadrado                   'aCua'
  13. Área de un triángulo                  'aTri'
  14. Área de un círculo                    'aCir'
  15. Suma de matrices (de enteros)         'sMat'
  16. Multiplicación de matrices (de enteros) 'mMat'
  17. Calcular un lote de operaciones. (sólo operaciones básicas, no matrices)
==============================================================================

 Seleccione una opción (0-17):
```

Figura 2: Menú de opciones de la calculadora.

En él se puede ver la gran cantidad y variedad de operaciones matemáticas que hay disponibles. Si introducimos 0 (`opcion = 0`), saldremos

de la aplicación. En otro caso, tras realizar la operación seleccionada con anterioridad, nos seguirá apareciendo el menú constantemente, por si queremos realizar más de una operación.

Además, tras realizar la operación se nos mostrará un mensaje diciendo `Pulse 'Enter' para continuar...`, y si lo pulsamos, nos volverá a aparecer el menú para volver a elegir otra operación. El motivo de hacerlo así es para no sobrecargar la interfaz o mejor dicho, la terminal, y que el usuario que utilice la calculadora no se sienta abrumado.

4.3.1. Ejemplo nº1. Operación básica. División

En este ejemplo vamos a realizar una división, vamos a dividir 44/35. Para ello seguimos los siguientes pasos:

1. Introducimos la opción nº 4 `opcion = 4`.
2. Después introducimos el primer operando (dividendo) 44 y pulsamos `'Enter'`.
3. A continuación introducimos el segundo operando (divisor) 35 y volvemos a pulsar `'Enter'`. Como vemos, se nos mostrará el resultado por pantalla.
4. Finalmente volvemos a pulsar `'Enter'` para que nos vuelva a aparecer el menú y poder continuar.

```
angelrf@angel-ubuntu:~/Practica_2_Llamada_a_procedimiento_remoto_RPC/Parte_2_Apache_Thrift/codigo$ python3 cliente.py
hacemos ping al server

==== CALCULADORA DISTRIBUIDA usando Apache Thrift ============================
Opciones disponibles:

  0. Salir de la aplicación.
  1. Suma                                   '+'
  2. Resta                                  '-'
  3. Multiplicación                         'x'
  4. División                               '/'
  5. Potencia                               '^'
  6. Raiz cuadrada                          'sqrt'
  7. Logaritmo base 10                      'log10'
  8. Logaritmo natural                      'ln'
  9. Seno                                   'sin'
  10. Coseno                                'cos'
  11. Tangente                              'tan'
  12. Área de un cuadrado                   'aCua'
  13. Área de un triángulo                  'aTri'
  14. Área de un círculo                    'aCir'
  15. Suma de matrices (de enteros)         'sMat'
  16. Multiplicación de matrices (de enteros) 'mMat'
  17. Calcular un lote de operaciones. (sólo operaciones básicas, no matrices)
================================================================================

 Seleccione una opción (0-17): 4
----------------------------------------
 Introduzca el 1er operando: 44
 Introduzca el 2o operando: 35
----------------------------------------

 >>> Resultado de 44.0 / 35.0 = 1.2571428571428571

 Pulse 'Enter' para continuar...
```

Figura 3: Ejemplo nº1. Dividir 44 entre 35, visto desde el lado del cliente.

En la siguiente imagen vemos un ejemplo de la excepción que se puede generar al intentar realizar una operación que no es válida, como por ejemplo, dividir entre 0.



Figura 4: Ejemplo nº1. Error al intentar dividir 44 entre 0, visto desde el lado del cliente.

Desde el lado del servidor (esta vez he ejecutado el servidor implementado en Python), se vería así:

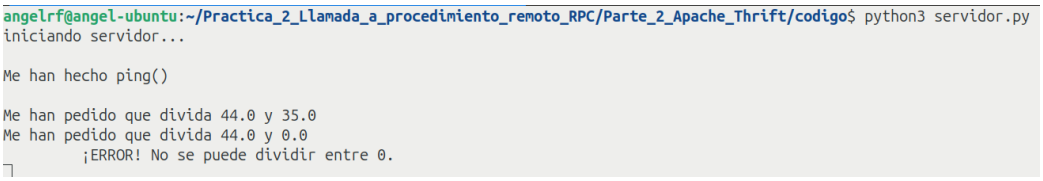


Figura 5: Ejemplo nº1 visto desde el lado del servidor.

4.3.2. Ejemplo nº2. Operación básica. Área de un Círculo.

En este ejemplo vamos a calcular el área de un triángulo con `base = 12.5` y `altura = 15.3` Para ello seguimos los siguientes pasos:

1. Introducimos la opción nº 13 `opcion = 13`.
2. Después introducimos la base del triángulo: 12,5 y pulsamos `'Enter'`.
3. A continuación introducimosla altura del triángulo: 15,3 y volvemos a pulsar `'Enter'`. Como vemos, se nos mostrará el resultado por pantalla.

4. Finalmente volvemos a pulsar `'Enter'` para que nos vuelva a aparecer el menú y poder continuar.

```
angelrf@angel-ubuntu:~/Practica_2_Llamada_a_procedimiento_remoto_RPC/Parte_2_Apache_Thrift/codigo$ python3 cliente.py
hacemos ping al server

==== CALCULADORA DISTRIBUIDA usando Apache Thrift ============================
Opciones disponibles:

  0. Salir de la aplicación.
  1. Suma                                   '+'
  2. Resta                                  '-'
  3. Multiplicación                         'x'
  4. División                               '/'
  5. Potencia                               '^'
  6. Raíz cuadrada                          'sqrt'
  7. Logaritmo base 10                      'log10'
  8. Logaritmo natural                      'ln'
  9. Seno                                   'sin'
  10. Coseno                                'cos'
  11. Tangente                              'tan'
  12. Área de un cuadrado                   'aCua'
  13. Área de un triángulo                  'aTri'
  14. Área de un círculo                    'aCir'
  15. Suma de matrices (de enteros)         'sMat'
  16. Multiplicación de matrices (de enteros) 'mMat'
  17. Calcular un lote de operaciones. (sólo operaciones básicas, no matrices)
================================================================================

 Seleccione una opción (0-17): 13
 ------------------------------------
 Introduzca la base del triángulo: 12.5
 Introduzca la altura del triángulo: 15.3
 ------------------------------------

 >>> Área de un triángulo de base 12.5 y altura 15.3 = 95.625

 Pulse 'Enter' para continuar...
```

Figura 6: Ejemplo nº2. Área de un Triángulo visto desde el lado del cliente.

Desde el lado del servidor (esta vez he ejecutado el servidor implementado en Java), se vería así:

```
angelrf@angel-ubuntu:~/Practica_2_Llamada_a_procedimiento_remoto_RPC/Parte_2_Apache_Thrift/codigo$ javac -d bin -cp ".:lib/*
" -sourcepath gen-java servidor.java
angelrf@angel-ubuntu:~/Practica_2_Llamada_a_procedimiento_remoto_RPC/Parte_2_Apache_Thrift/codigo$ java -cp "bin:gen-java:li
b/*" servidor
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Iniciando el servidor ...
Recibido ping()
Realizando área de un triángulo con base 12.5 y altura 15.3
```

Figura 7: Ejemplo nº2 visto desde el lado del servidor.

4.3.3. Ejemplo nº3. Operación con matrices

En este ejemplo vamos a calcular la **suma de dos matrices** de dimensiones `2 x 3`. Los pasos a seguir son:

1. Introducimos la opción nº 15 `opcion = 15`.
2. Introducimos el número de filas de la 1ª matriz.
3. Después el número de columnas de la 1ª matriz.
4. A continuación introducimos la primera fila de la 1ª matriz, **los elementos tienen que ir separados por un espacio**. Al terminar pulsamos`'Enter'`.
5. Repetimos la misma operación anterior para la segunda fila de la 1ª matriz. Al terminar pulsamos`'Enter'` y se nos mostrará como ha quedado la 1ª matriz.
6. Introducimos el número de filas de la 2ª matriz. (Debe ser el mismo que el de la 1ª matriz)
7. Después el número de columnas de la 2ª matriz. (Debe ser el mismo que el de la 1ª matriz)
8. Ahora introducimos la primera fila de la 2ª matriz, los elementos tienen que ir separados por un espacio. Al terminar pulsamos`'Enter'`.
9. Repetimos la misma operación anterior para la segunda fila de la 2ª matriz,los elementos tienen que ir separados por un espacio. Al terminar pulsamos`'Enter'` y se nos mostrará como ha quedado la 2ª matriz.
10. Nos aparecerá la matriz resultado de realizar la operación.
11. Finalmente volvemos a pulsar `'Enter'` para que nos vuelva a aparecer el menú y poder continuar.

```
angelrf@angel-ubuntu:~/Practica_2_Llamada_a_procedimiento_remoto_RPC/Parte_2_Apache_Thrift/codigo$
hacemos ping al server

==== CALCULADORA DISTRIBUIDA usando Apache Thrift ============================
Opciones disponibles:

  0. Salir de la aplicación.
  1. Suma                                   '+'
  2. Resta                                  '-'
  3. Multiplicación                         'x'
  4. División                               '/'
  5. Potencia                               '^'
  6. Raíz cuadrada                          'sqrt'
  7. Logaritmo base 10                      'log10'
  8. Logaritmo natural                      'ln'
  9. Seno                                   'sin'
  10. Coseno                                'cos'
  11. Tangente                              'tan'
  12. Área de un cuadrado                   'aCua'
  13. Área de un triángulo                  'aTri'
  14. Área de un círculo                    'aCir'
  15. Suma de matrices (de enteros)         'sMat'
  16. Multiplicación de matrices (de enteros) 'mMat'
  17. Calcular un lote de operaciones. (sólo operaciones básicas, no matrices)
================================================================================

 Seleccione una opción (0-17): 15

 Para esta operación, debe introducir dos matrices de enteros.
 Primero introducirá la primera matriz, luego la segunda.
 Asegúrese de que las dimensiones de las matrices sean compatibles con la operación seleccionada.
----------------------------------------------------------------

 Introduzca el número de filas de la matriz: 2
 Introduzca el número de columnas de la matriz: 3

 Introduzca los elementos fila por fila, separados por espacios, (total 3 elementos por fila):
 Fila 1: 1 2 3
 Fila 2: 4 5 6

 La primera matriz introducida es:
       [   1    2    3 ]
       [   4    5    6 ]
```

Figura 8: Imagen 1/2. Ejemplo nº3. Suma de matrices visto desde el lado del cliente.

```
 Introduzca la segunda matriz:
 Introduzca el número de filas de la matriz: 2
 Introduzca el número de columnas de la matriz: 3

 Introduzca los elementos fila por fila, separados por espacios, (total 3 elementos por fila):
 Fila 1: 7 8 9
 Fila 2: 10 11 12

 La segunda matriz introducida es:
       [   7    8    9 ]
       [  10   11   12 ]
----------------------------------------------------------------


 >>> Resultado de la operación con matrices:

--- MATRIZ 1 ---
       [   1    2    3 ]
       [   4    5    6 ]

      +

--- MATRIZ 2 ---
       [   7    8    9 ]
       [  10   11   12 ]

 = RESULTADO =
       [   8   10   12 ]
       [  14   16   18 ]

---------------
```

Figura 9: Imagen 2/2. Ejemplo nº3. Suma de matrices visto desde el lado del cliente.

Desde el lado del servidor, se vería así:

```
angelrf@angel-ubuntu:~/Practica_2_Llamada_a_procedimiento_remoto_RPC/Parte_2_Apache_Thrift/codigo$ java -cp "bin:gen-java:lib/*" servidor
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Iniciando el servidor ...
Recibido ping()
Me han pedido que sume dos matrices
 --- MATRIZ 1 ---
       [   1    2    3 ]
       [   4    5    6 ]
 --- MATRIZ 2 ---
       [   7    8    9 ]
       [  10   11   12 ]
 --- RESULTADO (MATRIZ SUMA) ---
       [   8   10   12 ]
       [  14   16   18 ]
```

Figura 10: Ejemplo nº3. Suma de matrices visto desde el lado del servidor.

4.3.4. Ejemplo nº4. Procesamiento de un lote de operaciones

. En este ejemplo vamos a calcular **un lote de 3 operaciones**. Los pasos a seguir son:

1. Introducimos la opción nº 17 `opcion = 17`.
2. Introducimos el número de operaciones que queremos realizar: `3` y pulsamos `'Enter'`.
3. Introducimos el operador de la 1ª operación. En este caso se trata de una raíz cuadrada: `sqrt`. Volvemos a pulsar `'Enter'`.
4. A continuación introducimos el primer operando (en este caso sería el radicando). Al terminar pulsamos `'Enter'`.
5. Repetimos los pasos nº3 y nº4 para la 2ª operación, que es el área de un cuadrado de lados `5.9`. Sería el operador `aCua`.
6. Repetimos los pasos nº3 y nº4 para la 3ª operación, que es el seno de `90` grados. Sería el operador `sin`.
7. A continuación nos aparecerán los resultados de todas las operaciones.
8. Finalmente volvemos a pulsar `'Enter'` para que nos vuelva a aparecer el menú y poder continuar.



Figura 11: Ejemplo nº4. Lote de operaciones visto desde el lado del cliente.

Desde el lado del servidor, se vería así:

```
angelrf@angel-ubuntu:~/Practica_2_Llamada_a_procedimiento_remoto_RPC/Parte_2_Apache_Thrift/codigo$ java -cp "bin:gen-java:lib/*" servidor
SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
Iniciando el servidor ...
Recibido ping()
Procesando operación: sqrt (5.0)
Realizando raíz cuadrada: sqrt(5.0)
Procesando operación: aCua (5.9)
Realizando área de un cuadrado con lado 5.9
Procesando operación: sin (90.0)
Realizando seno de 90.0 grados
```

Figura 12: Ejemplo nº4. Lote de operaciones visto desde el lado del servidor.